

JavaScript Übungsaufgaben

10 neue Aufgaben · alle Konzepte kombiniert · mit Erklärungen

Alle 10 Aufgaben bauen auf den Konzepten aus den vorherigen Übungen auf und führen neue Themen ein. Zu jeder Aufgabe gibt es einen Erklärungstext, der das Konzept einordnet, und einen Tipp.

Aufgabe 1: Ausgaben-Tracker

Themen: `localStorage` `reduce()` `filter()` Objekte DOM

■ **Einordnung:** Diese Aufgabe kombiniert Datenpersistenz mit aggregierender Auswertung. Der Kern liegt in `reduce()`, das aus einer flachen Liste ein Zusammenfassungs-Objekt baut – ähnlich wie eine `GROUP-BY`-Abfrage in SQL.

Baue einen Ausgaben-Tracker, der Geldausgaben nach Kategorien erfasst und dauerhaft speichert. Die App soll jederzeit zeigen, wie viel in welcher Kategorie ausgegeben wurde.

Teilaufgaben:

- 1.1 Formular mit drei Feldern: **Beschreibung**, **Betrag** (Dezimalzahl), **Kategorie** (Dropdown: Essen, Transport, Freizeit, Sonstiges). Speichere jede Ausgabe als Objekt `{ id, beschreibung, betrag, kategorie }`.
- 1.2 Zeige alle Ausgaben als Liste an, neueste zuerst. Jede Zeile zeigt Beschreibung, Betrag (formatiert mit `toFixed(2)`) und Kategorie. Löschen-Button pro Eintrag.
- 1.3 Berechne mit `reduce()` die Gesamtsumme aller Ausgaben und zeige sie prominent an.
- 1.4 Zeige darunter die Summe pro Kategorie – nur Kategorien anzeigen, für die Ausgaben vorhanden sind. Sortiere nach Betrag (höchste zuerst).

■ **Tipp Kategorie-Summen:** Baue mit `reduce()` ein Objekt auf: `{ Essen: 45.50, Transport: 12.00 }`. Dann `Object.entries(obj).sort((a,b) => b[1]-a[1])` zum Sortieren.

Aufgabe 2: Stoppuhr mit Runden

Themen: `setInterval` `clearInterval` `Date.now()` `Array` `DOM`

■ **Einordnung:** `Date.now()` gibt die aktuelle Zeit als Millisekunden seit 1970 zurück. Die verstrichene Zeit ist immer `Date.now() - startZeit`. Das ist robuster als einen Zähler hochzuzählen, weil es unabhängig davon ist, wie schnell der Browser das Interval ausführt.

Baue eine Stoppuhr, die Millisekunden genau misst und Rundenzeiten aufzeichnet. Die Uhr soll im laufenden Betrieb Runden erfassen können.

Teilaufgaben:

2.1 Buttons: Start, Stopp/Weiter, Runde, Reset. Eine Anzeige zeigt die laufende Zeit im Format `MM:SS.mm an`.

- **2.2** Starte die Zeit mit `Date.now()` als Referenz. Berechne die angezeigte Zeit immer als `Date.now() - startZeit`. Aktualisiere die Anzeige alle 10ms mit `setInterval`.
- **2.3** Der Runden-Button erfasst die aktuelle Zeit als Rundenzeit. Zeige alle Runden als Liste an: Runden-Nummer und absolute Zeit.
- **2.4** Reset setzt alles zurück: Zeit auf 0, Rundenliste leeren. Stopp/Weiter pausiert und setzt die Messung fort, ohne die Zeit zu verlieren.

■ **Tipp Pause:** Beim Pausieren die bisher vergangene Zeit als `offsetZeit` speichern. Beim Fortsetzen: `startZeit = Date.now() - offsetZeit`. So läuft die Uhr nach der Pause nahtlos weiter.

Aufgabe 3: Vokabeltrainer mit Bewertung

Themen: Klassen `localStorage` `Math.random()` `filter()` `map()`

■ **Einordnung:** Dieses "Spaced Repetition"-Prinzip steckt hinter Apps wie Anki. Falsch beantwortete Karten werden häufiger gezeigt, richtig beantwortete seltener. Die Klasse `Vokabel` kapselt diese Logik sauber.

Baue einen Vokabeltrainer mit einem Bewertungssystem. Jede Vokabel bekommt eine Schwierigkeitsstufe, die sich je nach Antwort anpasst.

Teilaufgaben:

- 3.1 Erstelle eine Klasse `Vokabel` mit den Feldern `id`, `original`, `uebersetzung`, `stufe` (1–5, Start: 3) und `richtigBeantwortet` (Zähler). Methode `bewerteAntwort(richtig)`: erhöht oder verringert `stufe`.
- 3.2 Formular zum Hinzufügen: Original-Wort und Übersetzung. Gespeichert als Array von `Vokabel`-Objekten im `localStorage`.
- 3.3 Übungs-Modus: zeige eine zufällige Vokabel (Original). Nutze `Math.random()`. Eingabefeld für die Antwort, Button "Prüfen". Vergleich ohne Groß-/Kleinschreibung.
- 3.4 Nach jeder Antwort: Bewertung anzeigen (richtig/falsch), Stufe der Vokabel aktualisieren, speichern. Zeige eine Statistik: Gesamtzahl, davon richtig beantwortet.

■ **Tipp Stufen:** Stufe 1 = sehr schwer (oft zeigen), Stufe 5 = leicht (selten zeigen). Bei richtig: `stufe = Math.min(5, stufe + 1)`. Bei falsch: `stufe = Math.max(1, stufe - 1)`.

Aufgabe 4: Drag-and-Drop Aufgabenliste

Themen: DOM Events dragstart dragover drop localStorage Array

■ **Einordnung:** Die Browser-eigene Drag-and-Drop-API nutzt Events: dragstart (Ziehen beginnt), dragover (über Element schweben), drop (loslassen). Die neue Reihenfolge wird im DOM sichtbar und gleichzeitig im Array und localStorage aktualisiert.

Baue eine Aufgabenliste, bei der Einträge per Drag-and-Drop umsortiert werden können. Die neue Reihenfolge soll dauerhaft gespeichert werden.

Teilaufgaben:

4.1 Normales Aufgaben-Array im localStorage: Aufgaben hinzufügen und als Liste anzeigen. Jeder Eintrag hat id und text.

- **4.2** Mache die Listenelemente draggable: `li.draggable = true`. Beim dragstart-Event: speichere die id der gezogenen Aufgabe in einer Variable.
- **4.3** Beim dragover-Event: `event.preventDefault()` aufrufen (sonst funktioniert drop nicht). Beim drop-Event: finde heraus, über welcher Aufgabe losgelassen wurde.
- **4.4** Beim Drop: tausche die Positionen der beiden Aufgaben im Array und speichere die neue Reihenfolge. Rendere die Liste neu.

■ **Tipp Drop-Logik:** Finde den Index beider Aufgaben im Array. Tausche sie: `const temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;`. Alternativ: Array neu aufbauen mit `arr.splice(j, 0, arr.splice(i, 1)[0])`.

Aufgabe 5: Rezept-Verwaltung mit Zutaten-Skalierung

Themen: Klassen `map()` `localStorage` Number DOM

■ **Einordnung:** Die Skalierung ist reines Verhältnisrechnen: $\text{Zutatenmenge} \times (\text{gewünschtePortionen} / \text{originalPortionen})$. Die ursprünglichen Mengen dürfen nie verändert werden – die Skalierung wird immer frisch berechnet.

Baue eine Rezept-Verwaltung. Rezepte können gespeichert und für eine beliebige Personenzahl skaliert werden: alle Zutatenmengen passen sich automatisch an.

Teilaufgaben:

- 5.1 Erstelle eine Klasse `Rezept` mit `id`, `name`, `portionen` und `zutaten` (Array von `{ name, menge, einheit }`). Methode `skaliere(neuePortionen)`: gibt neues Zutaten-Array zurück.
- 5.2 Formular: Rezeptname, Portionenzahl, und ein dynamisches Formular für Zutaten (Button "+ Zutat" fügt ein neues Eingabe-Trio hinzu: Name, Menge, Einheit).
- 5.3 Zeige alle gespeicherten Rezepte als Karten an. Ein Klick öffnet das Rezept in der Detail-Ansicht.
- 5.4 In der Detail-Ansicht: ein Eingabefeld für die gewünschte Portionenzahl. Bei jeder Änderung werden alle Zutatenmengen neu berechnet und angezeigt – mit `map()` und `skaliere()`.

■ **Tipp `skaliere()`:** `return this.zutaten.map(z => ({ ...z, menge: (z.menge * neuePortionen / this.portionen).toFixed(1) })).` *Spread (...) kopiert alle Felder, nur menge wird überschrieben.*

Aufgabe 6: Event-Bus (Publish-Subscribe)

Themen: Closure Higher-Order Function Map Callbacks Entkopplung

■ **Einordnung:** Das Pub-Sub-Muster ist in großen Anwendungen allgegenwärtig (z.B. als EventEmitter in Node.js). Eine Komponente "veröffentlicht" ein Ereignis, andere "abonnieren" es und reagieren darauf – ohne dass die erste Komponente die anderen kennen muss.

Implementiere ein Publish-Subscribe-System (Event-Bus). Damit können verschiedene Teile einer App miteinander kommunizieren, ohne sich direkt zu kennen.

Teilaufgaben:

- 6.1** Schreibe eine Funktion `createEventBus()`, die ein Objekt mit drei Methoden zurückgibt: `on(ereignis, callback)`, `off(ereignis, callback)`, `emit(ereignis, daten)`. Nutze intern eine Map für die Zuordnung.
- **6.2** `on()` registriert einen Listener für ein Ereignis. Mehrere Listener pro Ereignis sollen möglich sein. `off()` entfernt einen spezifischen Listener wieder.
 - **6.3** `emit()` ruft alle registrierten Listener für das genannte Ereignis auf und übergibt ihnen die Daten.
 - **6.4** Demonstriere den Event-Bus in einer kleinen App: Eine Funktion `benutzerAngemeldet(name)` emittiert ein Event. Zwei unabhängige "Komponenten" reagieren darauf: eine zeigt eine Begrüßung, eine andere loggt den Zeitpunkt.

■ **Tipp Map:** `const listeners = new Map().listeners.set("click", [fn1, fn2])` speichert ein Array. Prüfe mit `listeners.has(ereignis)` ob schon Listener existieren.

Aufgabe 7: Async Datenabruf mit Ladeindikator

Themen: `async/await` `fetch()` `Promise` `try/catch` `DOM`

■ **Einordnung:** Die öffentliche API `https://jsonplaceholder.typicode.com/posts` gibt JSON-Daten zurück. `fetch()` gibt ein `Promise` zurück, `.json()` darauf auch – deshalb zwei `await`s. Das Lademuster `Loading/Success/Error` ist ein Standard-Pattern in jeder modernen Webanwendung.

Baue eine App, die echte Daten von einer öffentlichen API lädt. Zeige während des Ladens einen Ladeindikator, handle Fehler sauber, und rendere die Daten übersichtlich.

Teilaufgaben:

7.1 Erstelle eine Funktion `async function datenLaden(url)`. Nutze `fetch(url)` und `await response.json()` um Daten zu laden. Wirf bei HTTP-Fehler einen eigenen Fehler (`response.ok` prüfen).

- **7.2** Zeige während des Ladens eine Ladeanzeige (`element.style.display = "block"`). Verstecke sie nach dem Laden – egal ob Erfolg oder Fehler (nutze `finally`).

- **7.3** Lade die ersten 10 Posts von `https://jsonplaceholder.typicode.com/posts?_limit=10`. Zeige für jeden Post Titel und Text in einer Karte an.

- **7.4** Füge ein Suchfeld hinzu, das die bereits geladenen Posts clientseitig filtert (kein neuer API-Aufruf). Speichere die Posts im Speicher, filtere mit `filter()`.

■ **Tipp finally:** `try { ... } catch(e) { ... } finally { ladeindikator.style.display = "none"; }`. `finally` läuft immer – egal ob Fehler oder nicht.

Aufgabe 8: Formular-Wizard (mehrstufiges Formular)

Themen: Klassen DOM localStorage Validierung Zustandsverwaltung

■ **Einordnung:** Wizard-Formulare sind ein UI-Pattern für komplexe Eingaben. Der Zustand (welcher Schritt, welche Daten) wird in einer Klasse gekapselt. Zwischenspeichern im localStorage verhindert Datenverlust bei versehentlichem Seitenverlassen.

Baue ein mehrstufiges Formular (Wizard) mit 3 Schritten. Jeder Schritt wird validiert, bevor man zum nächsten gelangt. Beim Verlassen der Seite werden eingegebene Daten zwischengespeichert.

Teilaufgaben:

8.1 Erstelle eine Klasse Formularwizard. Schritt 1: Name + E-Mail. Schritt 2: Adresse (Straße, PLZ, Stadt). Schritt 3: Zusammenfassung + Absende-Button. Immer nur ein Schritt ist sichtbar.

- **8.2** Methode `naechsterSchritt()`: validiert den aktuellen Schritt. Alle Pflichtfelder müssen ausgefüllt sein, E-Mail muss @ enthalten. Zeige Fehler direkt beim Feld an.
- **8.3** Methode `vorherigerSchritt()`: zurückgehen ohne Datenverlust. Eine Fortschrittsanzeige zeigt den aktuellen Schritt (z.B. "Schritt 2 von 3").
- **8.4** Speichere die eingegebenen Daten nach jeder Änderung im localStorage (`input.oninput`). Beim Laden der Seite werden die Daten wiederhergestellt.

■ **Tipp Sichtbarkeit:** Alle Schritt-Divs ausblenden, dann nur den aktiven einblenden: `schritte.forEach(s => s.style.display = "none"); schritte[aktuellerSchritt].style.display = "block".`

Aufgabe 9: Eigener Array-Helfer (Polyfill)

Themen: **prototype Array Higher-Order Function Closures Callbacks**

■ **Einordnung:** Eigene Implementierungen von eingebauten Methoden nennt man "Polyfills". Sie werden genutzt, um neue Features in alten Browsern verfügbar zu machen. Das Verstehen der Implementierung hilft enormt beim Verständnis, wie man diese Methoden effektiv einsetzt.

Implementiere `map()`, `filter()` und `reduce()` komplett selbst – ohne die eingebauten Array-Methoden zu nutzen. Das zeigt, wie diese Methoden intern funktionieren.

Teilaufgaben:

- 9.1 Implementiere `meinMap(array, callback)`: geht jedes Element durch, ruft `callback(element, index)` auf, sammelt die Rückgabewerte in einem neuen Array. Das originale Array darf nicht verändert werden.
- 9.2 Implementiere `meinFilter(array, callback)`: behält nur Elemente, bei denen `callback(element, index)` `true` zurückgibt.
- 9.3 Implementiere `meinReduce(array, callback, startwert)`: führt `callback(akkumulator, element, index)` für jedes Element aus. Der Startwert ist der initiale Akkumulator.
- 9.4 Teste alle drei Funktionen mit denselben Arrays wie die eingebauten Methoden und vergleiche die Ergebnisse. Bonus: Füge `meinForEach` und `meinFind` hinzu.

■ **Tipp Schleife:** Alle drei nutzen intern eine normale `for`-Schleife: `for (let i = 0; i < array.length; i++) { ... }`. Der Unterschied liegt darin, was mit dem Rückgabewert von `callback` gemacht wird.

Aufgabe 10: Mini-Framework: Reaktive UI

Themen: Proxy Closure DOM Getter/Setter Reaktivität

■ **Einordnung:** Proxy ist ein JavaScript-Objekt, das einen anderen Objekt "umhüllt" und Zugriffe abfangen kann. Wenn jemand eine Eigenschaft setzt (set-Trap), kann man darauf reagieren – z.B. die UI neu rendern. Genau das tun Vue.js und andere Frameworks intern.

Baue ein Mini-Framework, das automatisch die UI aktualisiert, wenn sich Daten ändern – ähnlich wie Vue.js oder React. Das Herzstück ist ein reaktiver Zustand mit Proxy.

Teilaufgaben:

10.1 Schreibe eine Funktion `reaktiv(daten, onChange)`. Sie soll ein Proxy-Objekt zurückgeben. Wenn eine Eigenschaft des Proxys gesetzt wird, wird automatisch `onChange()` aufgerufen.

- **10.2** Erstelle einen reaktiven Zustand für eine einfache Todo-App: `const zustand = reaktiv({ todos: [], filter: "alle" }, render)`. Jede Änderung an `zustand` löst `render()` aus.

- **10.3** `render()` liest den aktuellen Zustand und baut die DOM-Liste komplett neu auf. Filter "alle", "offen", "erledigt" funktionieren durch Ändern von `zustand.filter`.

- **10.4** Füge hinzu: Todo hinzufügen setzt `zustand.todos = [...zustand.todos, neues]` – das löst den Proxy-Trap aus und `render()` wird automatisch aufgerufen. Kein manuelles Aufrufen von `render()` nötig.

■ **Tipp Proxy:** `new Proxy(daten, { set(obj, key, value) { obj[key] = value; onChange(); return true; } })`. Das `return true` ist Pflicht – es signalisiert, dass das Setzen erfolgreich war.